Assignment 4

For Assignment 4, we reviewed each other's code and identified areas where we could improve clarity, structure, and maintainability. This first page covers the eight improvements I (Sunny) made to Noah's code. Each change was applied and committed on a separate branch and then merged into the Assignment 4 branch after testing to ensure nothing broke. The sections below explain each improvement and why it was helpful. Noah's review of my code appears on the next page.

The first issue I found in my partner's code was that GameFactory created its own MazeLoader and Random objects inside the class. This made the code harder to test and hid the actual dependencies. I refactored the class so these objects are passed in through constructors, which keeps the design cleaner while keeping the default behaviour the same.

Another area for improvement was that the dynamic map update logic (refreshDynamicMap) was placed in GameFactory, even though it dealt entirely with the state of the World. To fix this, I moved the method into the World class so that the world manages its own visual representation. This increases cohesion and makes the design more consistent.

After moving the method, the old static version in GameFactory still existed and duplicated the behaviour. To avoid this, I kept the static method but turned it into a simple wrapper that calls world.refreshDynamicMap(). This removes duplication while keeping existing calls in the codebase working normally.

In configureWorld, I noticed a line that attempted to remove exit tiles from the open tile list. Exit tiles were never added to that list in the first place, which made the line unnecessary and confusing. I removed it to clean up the method and make its behaviour easier to understand.

I also found leftover debugging output in MazeLoader that printed the maze filename every time a new maze was loaded. This wasn't needed in the final game and cluttered the console, so I removed the print statement to keep the output clean.

Inside the maze-loading logic, the row-length and row-count checks were written in line and repeated. This made the method longer than it needed to be, so I extracted these checks into small helper methods. This makes the code easier to read and reduces duplication.

In GameFactory, the distanceSquared method didn't rely on any instance data but was still written as an instance method. I changed it to a static method and added a short comment explaining its purpose, so it's clearer that it works as a simple utility function.

My final improvement was that MazeLoader didn't validate the characters inside the maze file, which could allow small typos or invalid characters to slip through. I added a new validateCell() method to ensure all characters in the maze file are valid. This improves the safety of the loader without affecting any existing mazes.

The first issue I (Noah) found was the naming of the start and stop functions was a little ambiguous. I renamed them to startEngineThread and stopEngineThread to make their purpose a little more clear

Another area of improvement was the hasCollectedAllFragments method
It used to return a long statement spread out over 2 lines. I refactored it to use two different appropriately named booleans (noFragmentsRequired and allFragmentsCollected) and returned an or statement of these two booleans. This improves readability and allows anyone to understand what the function is doing at just a glance.

I also removed the unused method getState. It was never called and after testing I determined it was dead code and not needed anymore.

I changed the method isWalkable to isWall and inverted the return method as the function was only ever being used to check if a space was not walkable (thus a wall).

I also changed the isWithinBounds method to be isOutOfBounds and changed its code to check if the target coordinates are outside of the map rather than checking if they are inside.

I removed an unneeded if statement in the startEngine thread function that checked if the engine was already running. The engine is never initialised more than once and is unaffected by a secondary instance of the game running at the same time. Thus the if statement was always false.

I also removed an if statement in update() that never was used because the only function that calls update() also checks the same value before calling it and various other functions.

I refactored a section of the update() function that checked if the player successfully moved and then ran more code if they did. I was able to make the function shorter and less costly by moving the additional code into where it set the playerMoved to true.

I also changed the way the update function handled medkits to combine two different sections into one. The part that checked if the player had used a medkit and the part that checked sent the message to the system were originally separate. I combined it all into one function to make it more readable and concise.